Crashkurs Programmieren

Herzlich Willkommen!

Tauche ein in die Welt des **Programmierens**.
Dieses Dokument bietet
Konzepte, Prinzipien, Denkanstöße.
für einen erfolgreichen Einstieg, egal wo du auf deiner persönlichen Coding-Journey stehst.

Disclaimer: Dieses Dokument wurde mit Hilfe von KI erstellt, Texte werden noch geprüft und ggf. angepasst.

Blättere, experimentiere und sei neugierig – beim Coden kann man immer wieder neue, bessere oder auch kreativere Wege finden.

Direkt zum Inhalt »

How to Programmieren

Inhaltsverzeichnis

1. Was ist Informatik?
2. Programmieren
    - Wir fangen an
    - Wie schreibe ich aber Pseudocode? (objektorientiert)
    - Yippie, ich verstehe Pseudocode, aber wie weiter?
    - Ist es normal, dass es so hässlich ausschaut tho?
    - Was mach ich jz mit dem Pseudocode?
3. Kontrollstrukturen
    - If-Else und Vergleichsoperatoren
    - Logische Operatoren
    - Short Circuit Evaluation
    - Switch-Case
    - Schleifen
4. Gängige Datentypen
    - Bool
    - Zahlen
    - String
    - List
5. Python und dessen Schizophrenie
6. C - my beloved
    - Eine *richtige* Programmiersprache
    - Speicherallokation und Pointer (Zeiger)
    - Exkurs: C++

Was ist Informatik?

```
Die Informatik beschreibt eine Ansammlung verschiedener Disziplinen angewandter Mathematik.
Ganz gleich ob Systeme konzeptioniert und entwickelt werden oder man einfach nur den Code-
Monkey macht.
```

Programmieren

```
Das Progammieren / Coden beschreibt das Definieren und Anwenden von Regeln, um einen
bestimmten Prozess "zum Leben zu erwecken"
```

Wir fangen an

Alles was existiert und jemals existieren wird ist eine Funktion, nutzt eine Funktion oder wird von einer Funktion genutzt.

```
Ein Apfel wird gegessen
Ein Mensch isst.
=> Ein Mensch isst einen apfel
```

Der erste Schritt ist genau das zu verstehen.
Der nächste Schritt ist dieses in korrekter Form aufzuschreiben.

```
essen(Apfel)
mensch.essen
=> Mensch.essen(Apfel)
```

Der obige Teil ist eine Darstellung in "Pseudo-Code", also keiner *richtigen* Programmiersprache.
Allerdings kürzt diese Schreibweise den Klartext schon soweit, dass er mit nur wenigen Veränderungen an die entsprechende Sprache angepasst werden kann.

Wie schreibe ich aber Pseudocode?

(objektorientiert)

```
Ich mache dinge
=> Ich.machen(Dinge)
Peter prüft Pinguine und gibt ein Pinguinprüfzertifikat aus
=> Pinguinprüfzertifikat = Peter.prüfen(Pinguine)
```

Die Handlung / Funktion gehört zum Objekt.
Deshalb schreiben wir

```
Objekt.handeln()
```

Das Subjekt, das von der Handlung betroffen ist, kommmt in die Klammer und wird der Funktion übergeben, also:

```
Objekt.handeln(Subjekt)
```

Das Subjekt wird in dem Fall als **Übergabeparameter** bezeichnet.

Das Ergebnis der Handlung wird in der Variable *links* vom Gleichheitszeichen geschrieben.

```
Variable = Objekt.handeln(Subjekt)
```

Damit das gelingt, muss die Funktion **handeln** einen Rückgabewert besitzen. Also einen Wert "returnen" / zurückgeben.

```
function handeln(...)
    ...
    return ergebnis
```

Yippie, ich verstehe Pseudocode, aber wie weiter?

Jetzt schauen wir nach, was wir eigentlich wollen.

Beispiel:

```
Gegeben sei eine vollautomatisierte Küche.
Progammieren Sie eine Routine, die einen Kuchen ausgibt.
```

Unsere Situation gibt uns vor, womit wir arbeiten können:

```
Vollautomatisierte Küche
```

Wenn uns kein weiterer Kontext gegeben wird, werden wir kreativ & behaupten einfach, dass die Dinge so existieren, wie wir sie haben wollen *(ähnlich wie in der Mathematik)*.

Dementsprechend "besorgen" wir uns:

```
Ofen
Trockenzutaten
Nasszutaten
Mischroboter
Schüsselroboter
Greifarm
Kuchenform
```

Und jetzt schreiben wir unseren Pseudocode:
```python
function Backroutine():
Schüssel1 = Schüsselroboter.NimmNeueSchüssel()
Schüssel2 = Schüsselroboter.NimmNeueSchüssel()

    Mischroboter.Mischen(Trockenzutaten,Schüssel1)
    Mischroboter.Mischen(Nasszutaten,Schüssel2)
    Schüssel1 = Mischroboter.SchüsselnMixen(Schüssel1,Schüssel2)
    Greifarm.KippInForm(Schüssel1,Kuchenform)
    Greifarm.BewegeInOfen(Kuchenform)
    Ofen.An()
    Solange Kuchenform.Inhalt nicht gebacken
        WartenInMinuten(1)
    Ofen.Aus()
    Greifarm.BewegeAusOfen(Kuchenform)

    return Kuchenform.Inhalt
    ```
```

Nach einem ersten groben Entwurf, prüfen wir, ob wir alles bedacht haben, was wir machen wollten:
```

```
"Gegeben sei [....]"
"progammieren Sie eine Routine"
"die einen Kuchen ausgibt"
```

- Wir haben uns von den vorgefertigten / gegebenen Sachen bedient *(trust)*
- wir haben eine seperate Funktion geschrieben, die als Routine von überall aufgerufen werden kann
- wir haben den Inhalt der Kuchenform ausgegeben, also den Kuchen.

Ist es normal, dass es so hässlich ausschaut tho?

Jaein, Pseudocode kann aussehen wieauchimmer mann will

Was mach ich jz mit dem Pseudocode?

Auf deine gewünschte Sprache anpassen.
Heißt:

Keywords anpassen.
Syntax anpassen, also Punkte, Kommata, Leerzeichen.
vgl. pseudocode vs python vs c#:

Note:
Man kann auch behaupten, dass irgendein Sklave die ganzen Aktionen durchführt, solange die Sachen passieren, ist es **(meistens)** irrelevant, wer genau das macht.

Pseudocode

```
function BrotBelegen(Brot brot, Brotbelag brotbelag)
    butter = neue Butter
    maggi = neue Maggi
    brot = maggi.tränken(brot)
    brot = butter.schmieren(brot)
    brot = brotbelag.legenAuf(brot)
    return brot
```

python

```python
def BrotBelegen(brot,brotbelag)
    butter = Butter()
    maggi = Maggi()
    brot = maggi.tränken(brot)
    brot = butter.schmieren(brot)
    brot = brotbelag.legenauf(brot)
    return brot
```

c#

```
Brot BrotBelegen(Brot brot,BrotBelag brotbelag)
    butter = new Butter();
    maggi = new Maggi();
    brot = maggi.tränken(brot);
    brot = butter.schmieren(brot);
    brot = brotbelag.legenauf(brot);
    return brot;
```

Wie wir sehen, ändert sich nicht viel.
Wir gehen allerdings von einigen Sachen aus:

```
Maggi hat Funktion Tränken
Tränken nimmt ein Brot und gibt das auch wieder zurück

Ebenso Butter.Schmieren() und BrotBelag.Legenauf()
```

Kontrollstrukturen

Kontrollstrukturen sind die primitivste Form, den Fluss des Programms zu kontrollieren.

Verzweigungen / if-Statements

Pseudocode

```
if Bedingung dann
    // tue etwas
sonst wenn Bedingung2 dann
    // tue etwas anderes
sonst
    //tue etwas ganz anderes
```

Beispiel (in Python):

```python
x = 5
if x > 5:
    print("x ist größer als 5")
elif x == 5:
    print("x ist genau 5")
else:
    print("x ist kleiner als 5")
```

Beispiel (in C#):

```csharp
int x = 5;
if (x > 5)
{
    Console.WriteLine("x ist größer als 5");
}
else if (x == 5)
{
    Console.WriteLine("x ist genau 5");
}
else
{
    Console.WriteLine("x ist kleiner als 5");
}
```

Besonderheiten

Besonderheiten beim Agieren mit if-Statements:

- **Reihenfolge und Kurzschlusslogik (Short Circuit Evaluation):**
  - Bei logischen Operatoren wie `and` (Python) bzw. `&&` (C#/Java):
    Der zweite Ausdruck wird **nur geprüft, wenn der erste wahr ist**.

```python
if x == y and y == z:
    # y == z wird nur überprüft, falls x == y True ist
```

```java
if (x == y && y == z) {
    // y == z wird nur geprüft, falls x == y true ist
}
```

  - Bei `or` (Python) bzw. `||` (C#/Java):
    Der zweite Ausdruck wird **nur geprüft, wenn der erste falsch ist**.

```python
if x == y or y == z:
    # y == z wird nur überprüft, falls x == y False ist
```

```java
if (x == y || y == z) {
    // y == z wird nur geprüft, falls x == y false ist
}
```

- **Vorsicht bei Funktionen/Ausdrücken mit Nebenwirkungen:**
  Wenn in einem Bedingungsteil z. B. eine Funktion aufgerufen wird, die etwas ausführt (z.B. eine Ausgabe macht oder einen Wert verändert), kann es sein, dass sie im Fall des Short Circuit gar nicht ausgeführt wird!
  - Beispiel (Python):

```python
def test():
    print("Test!")
    return True

if True or test():
    pass  # test() wird NICHT ausgeführt, da True schon reicht
```

- **Mehrere Bedingungen hintereinander:**
  Mehrere verschachtelte oder aufeinanderfolgende `if`, `elif` und `else`-Blöcke werden in der angegebenen Reihenfolge geprüft. Sobald eine zutrifft, werden die weiteren ignoriert (bei elif/else-Ketten).

- **Vergleich von mehreren Werten:**
  In einigen Sprachen kann man sogenannte Kettenvergleiche durchführen:
  - Python:

    ```
    if 3 < x < 10:
        # Entspricht: (3 < x) and (x < 10)
    ```

  - In anderen Sprachen muss man das explizit mit `&&` verknüpfen.

- **Typenkonversionen:**
  Achtung!
  Theoretisch lassen sich alle Objekte miteinander vergleichen, allerdings ist es nicht immer sinnvoll!

  ```
  if 3 < x < "Sinnloser Vergleich"
      ...
  ```

Switch-Case

Für den Fall, dass sich if-statements häufen:

```
switch Wert
    fall 1:
        // tue etwas
    fall 2:
        // tue etwas anderes
    sonst:
        // Standardfall
```

Beispiel (in Python mit match-case ab Version 3.10):

```
wetter = "sonnig"

match wetter:
    case "sonnig":
        print("Es ist schön draußen.")
    case "regnerisch":
        print("Regenschirm mitnehmen!")
    case _:
        print("Unbekanntes Wetter.")
```

Beispiel (in C#):

```
string wetter = "sonnig";

switch (wetter)
{
    case "sonnig":
        Console.WriteLine("Es ist schön draußen.");
        break;
    case "regnerisch":
        Console.WriteLine("Regenschirm mitnehmen!");
        break;
    default:
        Console.WriteLine("Unbekanntes Wetter.");
        break;
}
```

Hinweis:

- In vielen Sprachen (wie C, C#, Java) ist ein `break;` nach jedem Fall nötig, um zu verhindern, dass die nachfolgenden Fälle automatisch ausgeführt werden!
- In Python heißt die entsprechende Struktur seit Version 3.10 `match-case` und ist moderner als das klassische `switch-case`.

Schleifen

Gängige Schleifenstrukturen

Schleifen dienen dazu, Anweisungen mehrfach hintereinander auszuführen, solange eine bestimmte Bedingung erfüllt ist.

Beispiele:

while-Schleife (Python):

Wiederholt, solange Bedingung wahr ist

```
x = 0
while x < 5:
    print("x ist", x)
    x += 1
```

for-Schleife (C#):
Wiederholt eine feste Anzahl von Durchläufen

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine("i: " + i);
}
```

for-each-Schleife (Python):
Geht alle Elemente einer Sammlung/Liste durch

```
farben = ["rot", "grün", "blau"]
for farbe in farben:
    print(farbe)
```

do-while-Schleife (Java):
Führt erst aus, prüft dann Bedingung (mind.1x)

```
int y = 0;
do {
    System.out.println("y ist " + y);
    y++;
} while (y < 3);
```

Hinweise:

- Die genauen Syntax-Details hängen von der Programmiersprache ab.
- Schleifen können mit `break` (Abbruch) oder `continue` (nächster Schleifendurchlauf) beeinflusst werden.
- Eine Schleife **braucht** ein Abbruchkriterium, ansonsten kann sich das Programm *aufhängen*

Gängige Datentypen

Gängige Datentypen

Bool

Logisches True/False.

Ausdruck	Bedeutung / True wenn...	Beispiel(e)
`wahr` / `true`	Wahrheitswert true	`if (true)`
`falsch` / `false`	Wahrheitswert false	`if (false)`
`a == b`	Gleichheit von a und b	`if (x == 5)`
`a != b`	Ungleichheit von a und b	`if (x != 0)`
`a > b`	a ist größer als b	`if (note > 3)`
`a < b`	a ist kleiner als b	`if (alter < 18)`
`a >= b`	a ist größer oder gleich b	`if (punkte >= 10)`
`a <= b`	a ist kleiner oder gleich b	`if (tage <= 7)`
`!a` / `not a`	Negation (nicht a)	`if (!fertig)`
`a && b` / `a and b`	Logisches UND	`if (x > 5 && y < 10)`
`a || b` / `a or b`	Logisches ODER	`if (hungrig || müde)`
Variable/Objekt	True, wenn nicht null/leer/0 (je nach Sprache)**	`if (text)` / `if (liste)`

Hinweis:
Der Wahrheitswert von Variablen kann sich in der Auswertung je nach Programmiersprache unterscheiden:

- In Python ist z. B. `0`, `''`, `[]`, `None` als false gewertet. Alles andere ist true.
- In C/C++ sowie C# ist `0` false, alles andere true.
- In Java sind nur boolesche Ausdrücke erlaubt; Objekte müssen explizit verglichen werden (z. B. `obj != null`).

Gängige BoolOperatoren

Die wichtigsten Operatoren für Verknüpfungen und Bedingungen:

Operator	Bedeutung	Beispiel
`&&`	Logisches UND	`if (a && b)`
`||`	Logisches ODER	`if (x > 1 || y < 0)`
`!`	Logische Negation	`if (!fertig)`
`^`	Exklusives ODER (XOR)	`if (x ^ y)` *(je nach Sprache)*

Hinweis:

- Mit `&&` müssen beide Bedingungen wahr sein, damit das Gesamtergebnis wahr ist.
- Mit `||` reicht es, wenn eine der Bedingungen wahr ist.
- Mit `!` kehrt man den Wahrheitswert um.
- Der Operator `^` steht in einigen Sprachen für exklusives ODER (nur wahr, wenn *genau eine* Bedingung wahr ist) – das ist aber nicht in jeder Sprache gleich!

Praktische Beispiele (C#/Java/Python):

```
if (x > 5 && y < 3)  // beide Bedingungen wahr
if (!fertig)         // wenn nicht fertig
if (a == 1 || b == 2) // eine oder beide Bedingungen wahr
```

```
if x > 5 and y < 3:
if not fertig:
if a == 1 or b == 2:
```

Integer & Float

Typ	Speicherplatz	Wertebereich
short	2 Byte	-32.768 bis 32.767
int	4 Byte	-2.147.483.648 bis 2.147.483.647
long	8 Byte	-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807
unsigned short	2 Byte	0 bis 65.535
unsigned int	4 Byte	0 bis 4.294.967.295
unsigned long	8 Byte	0 bis 18.446.744.073.709.551.615

Typ	Speicherplatz	Wertebereich (ungefähr)
float	4 Byte	$\sim 1{,}5 \times 10^{-45}$ bis $3{,}4 \times 10^{38}$
double	8 Byte	$\sim 5{,}0 \times 10^{-324}$ bis $1{,}7 \times 10^{308}$

Hinweis: Die genauen Werte können je nach Programmiersprache und Plattform leicht variieren.

Gängige Operatoren für int & float:

Operator	Bedeutung	Beispiel
+	Addition	5 + 3 = 8
-	Subtraktion	5 - 3 = 2
*	Multiplikation	5 * 3 = 15
/	Division	5 / 2 = 2 (Ganzzahl), 5.0 / 2 = 2.5 (Float)
%	Modulo (Restwert der Division)	5 % 2 = 1
++	Inkrement (um 1 erhöhen)	i++ // i wird 1 größer
--	Dekrement (um 1 verringern)	i-- // i wird 1 kleiner
+=	Addiere und weise zu	i += 2 // entspricht i = i + 2
-=	Subtrahiere und weise zu	i -= 2 // entspricht i = i - 2

Hinweis:

- Der `/`-Operator liefert bei ganzen Zahlen (int) die ganzzahlige Division, bei Gleitkommazahlen (float/double) das tatsächliche Ergebnis mit Nachkommastellen.
- Operatoren wie `++` und `--` existieren in vielen Programmiersprachen, aber nicht in allen (z.B. Python nicht).

String

Eine schematische Darstellung eines Strings im Speicher (z. B. `"Hallo"`):

Index	0	1	2	3	4	5
Wert	H	a	l	l	o	\0*

*`\0` bezeichnet den "Null-Terminator" und wurde früher genutzt, um das Ende des Strings zu kennzeichnen. In moderneren Sprachen wie Python oder Java merkt sich das System die Länge des Strings anderweitig und ein solcher Abschluss ist oft nicht sichtbar.

- **Jeder Buchstabe** des Strings wird als eigenes Zeichen (z. B. als Zahl im Zeichensatz wie ASCII oder UTF-8) gespeichert.
- In vielen Programmiersprachen ist ein String einfach eine Kette (engl. "chain") von Zeichen, die hintereinander im Speicher liegen.

Gängige Operatoren für string:

Operator	Bedeutung	Beispiel
+	Verkettung von Strings ("concatenation")	"Hallo" + " Welt" = "Hallo Welt"
+=	Anhängen an einen String und Zuweisung	s += "!" // aus "abc" wird "abc!"
[] oder charAt	Zugriff auf einzelnes Zeichen	"Hallo"[1] = 'a' bzw. "Hallo".charAt(1)
len(), length	Länge des Strings ermitteln	len("Hallo") = 5 bzw. "Hallo".length()
in	Überprüfen, ob Teilstring enthalten	"ll" in "Hallo" → True (Python)
==	Vergleich auf Gleichheit	"abc" == "abc" → True
<, >, compareTo	lexikographischer Vergleich	"abc" < "def" → True / "abc".compareTo("def") < 0

Hinweis:

- Die genaue Syntax und verfügbare Operatoren/Methoden können sich je nach Programmiersprache unterscheiden!
- Viele Sprachen bieten zusätzliche nützliche Methoden wie `.replace()`, `.lower()`, `.upper()`, `.strip()`, `.split()`, usw.

List

Eine schematische Darstellung einer Liste im Speicher (z. B. `[10, 20, 30]`):

Index	0	1	2
Wert	10	20	30

- Eine **Liste** (engl. "list", auch Array, Feld, Vektor je nach Sprache) speichert mehrere Werte geordnet hintereinander.
- Jeder Wert ist über seinen **Index** (beginnt meist bei 0) erreichbar:
  Zum Beispiel liefert `liste[1]` das Element `20` (zweites Element).
- Listen können in den meisten Sprachen beliebig viele Elemente enthalten, oft sogar unterschiedliche Typen (z. B. in Python).
- In streng typisierten Sprachen wie Java/C++ muss meist vorher festgelegt werden, welcher Typ die Listenelemente haben.

Gängige List-Arten:

(Fortgeschritten)

Listenart	Beschreibung	Beispiel (Python)
Einfache Liste / Array	Feste oder dynamische Folge von Werten, Zugriff per Index	[1, 2, 3]
Dynamische Liste	Größe kann zur Laufzeit wachsen oder schrumpfen	list.append(4)
Verkettete Liste	Jedes Element verweist auf das nächste Element (seltener in Hochsprachen)	custom: Node(1) → Node(2)
Stack (Stapel)	LIFO-Prinzip ("Last In, First Out"), Einfügen/Entfernen am Ende	stack.append(x), pop()
Queue (Warteschlange)	FIFO-Prinzip ("First In, First Out"), Einfügen hinten, Entfernen vorne	queue.append(x), pop(0)
Set (Menge)	Nur eindeutige Werte, Reihenfolge oft egal	set([1, 2, 3])
Dictionary / Map	Speicherung von Schlüssel-Wert-Paaren, kein Zugriff über Index	{"a": 1, "b": 2}

- Welche Listenart verwendet wird, hängt von der Programmiersprache und der benötigten Funktionalität ab.
- In Python heißen Listen einfach `list`, in Java meist `ArrayList` oder `LinkedList`, in C/C++ oft `Array` oder spezielle Klassen (z. B. `std::vector`).

Gängige Listoperatoren:

Operator / Methode	Bedeutung	Beispiel (Python)
[]	Zugriff auf Element via Index	liste[2] gibt das dritte Element zurück
=	Zuweisung eines Elements	liste[0] = 17
+, +=	Konkatenation/Erweiterung von Listen	liste + [100] / liste += [200]
*	Mehrfache Wiederholung der Liste	[1,2]*3 ergibt [1,2,1,2,1,2]
append()	Hängt Element ans Listenende an	liste.append(42)
insert()	Fügt Element an bestimmter Stelle ein	liste.insert(1, 99)
extend()	Hängt mehrere Elemente an	liste.extend([7,8])
pop()	Entfernt ein Element (und gibt es zurück)	liste.pop() / liste.pop(0)
remove()	Entfernt das erste Vorkommen eines Werts	liste.remove(20)
del	Löscht Element oder ganzen Bereich	del liste[2] / del liste[1:3]
len()	Gibt die Länge der Liste zurück	len(liste)
in	Prüft, ob Wert enthalten ist	5 in liste
index()	Liefert den Index eines ersten Auftretens	liste.index(20)
count()	Zählt, wie oft ein Wert in der Liste ist	liste.count(10)
sort() / reverse()	Sortiert/Kehrt Reihenfolge um	liste.sort(), liste.reverse()
copy()	Erstellt eine (flache) Kopie	liste.copy()

Hinweise:

- Die genaue Syntax oder verfügbare Operatoren/Methoden hängt von der Sprache ab.
- In manchen Sprachen (z. B. Java) gibt es abweichende Methoden wie `add()`, `get()`, `set()`, `remove()` etc.
- Oft sind Listen wandelbar (mutable), Arrays in manchen Sprachen können jedoch unveränderbar (fixed size) sein.

Dictionary (Schlüssel-Wert-Paare)

In Python zum Beispiel:

```
person = {
    "name": "Alice",
    "alter": 25,
    "email": "alice@example.com"
}
```

- Jeder **Schlüssel** ist eindeutig und verweist auf einen bestimmten **Wert**.
- Zugriff auf einen Wert per Schlüssel: `person["alter"] - ergibt 25`
- Man kann beliebige Datentypen für Werte verwenden (Zahlen, Listen, weitere Dictionaries usw.).

Gängige Operatoren und Methoden für Dictionaries:

Operator / Methode	Bedeutung	Beispiel (Python)
[]	Zugriff auf Wert mittels Schlüssel	person["name"] gibt "Alice" zurück
=	Hinzufügen oder Überschreiben eines Werts	person["ort"] = "Berlin"
del	Entfernt einen Eintrag	del person["alter"]
in	Prüft, ob Schlüssel existiert	"name" in person
get()	Gibt Wert zu Schlüssel, sonst Default	person.get("name") / person.get("handy", "n.v.")
keys()	Liefert alle Schlüssel als Liste	person.keys()
values()	Liefert alle Werte als Liste	person.values()
items()	Liefert alle Schlüssel-Wert-Paare	person.items()
pop()	Entfernt Schlüssel und gibt Wert zurück	person.pop("email")
update()	Aktualisiert dict mit Einträgen	person.update({"plz": 12345})
clear()	Entfernt alle Einträge	person.clear()
copy()	Gibt Kopie des dict zurück	person.copy()

Hinweise:

- Schlüssel müssen unveränderbar (z. B. String, Zahl, Tupel) sein.
- Mit `for k, v in person.items():` lassen sich alle Paare durchlaufen.

Python und dessen Schizophrenie

Python gilt auf den ersten Blick als sehr klare und einsteigerfreundliche Programmiersprache, bringt jedoch einige Besonderheiten und teils überraschende Verhaltensweisen mit, die man kennen sollte.

Denn:
In Python sprechen wir Dinge einfach so into Existence.
Egal wo, egal wann.

Wenn ich einen Apfel habe, kann ich ihm jederzeit ein *.label = "Banane"* geben und niemand kann mich dran hindern. Auch wenn du eigentlich Kupfer erwartest, kann ich dir ohne Sorgen Pflastersteine hinwerfen, sofern du es nicht **extra** prüfst.

- **Indirekte Typisierung:** Variablen haben keinen festen Typ. Man kann z.B. erst einen Integer, dann einen String zuweisen. Beispiel:

```
a = 5
a = "Hallo"
```

- **Mutierbare Standardwerte in Funktionen:** Verwendet man z.B. eine Liste als Default-Argument, „trennt" Python dieses Default-Objekt nicht pro Funktionsaufruf ab:

```
def append_item(x, lst=[]):
    lst.append(x)
    return lst

print(append_item(1))  # [1]
print(append_item(2))  # [1, 2] !!! (nicht erneut [2])
```

- **Integer sind beliebig groß:** Anders als in vielen anderen Sprachen wachsen Zahlen „nach oben" ohne Überlauf.

- **Keine privaten Attribute:** In Python gibt es keine echten privaten Attribute, sondern Sichtbarkeit ist relativ zum Scope. Alles ist zugreifbar, aber über Namenskonventionen (_foo, __bar) signalisiert man, was "intern" gedacht ist.

```
class MeineKlasse:
    def __init__(self):
        self.oeffentlich = 1     # öffentlich zugreifbar
        self._intern = 2         # Konvention: "intern"
        self.__ganz_intern = 3   # Namens-Mangling: "_MeineKlasse__ganz_intern"

obj = MeineKlasse()
print(obj.oeffentlich)      # 1
print(obj._intern)          # 2
print(obj._MeineKlasse__ganz_intern)  # 3
```

- **Duck Typing** Hauptsache, ein Objekt „kann“ die gewünschte Methode/Operation, der tatsächliche Typ ist meist egal.

```
def verdoppeln(x):
    return x * 2

print(verdoppeln(3))         # 6 (int)
print(verdoppeln("Hi"))      # "HiHi" (str)
print(verdoppeln([1, 2]))    # [1, 2, 1, 2] (list)
```

- **Indentation zählt:** Blöcke werden durch Einrückung, nicht durch Klammern definiert.

- **Listen, dicts & Co. sind Referenztypen:** Zuweisung macht KEINE Kopie des Objekts. Beispiel:

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
```

- **Eigenschaften außerhalb des `__init__` zuweisen:** In Python kann man einer Klasseninstanz jederzeit neue Attribute außerhalb des Konstruktors (`__init__`) hinzufügen. Das macht die Sprache sehr flexibel, kann aber auch zu schwer zu findenden Fehlern führen.

```
class MeineKlasse:
    def __init__(self):
        self.x = 1

obj = MeineKlasse()
obj.y = 2   # neues Attribut außerhalb des inits!
print(obj.x)  # 1
print(obj.y)  # 2

obj2 = MeineKlasse()
print(hasattr(obj2, 'y'))  # False
```

  Das heißt, Instanzen derselben Klasse können unterschiedliche Attribute besitzen – „*sc*hizoph*re*n“!

Diese „Schizophrenien“ führen manchmal zu merkwürdigen Bugs – deshalb: Verständnis zahlt sich aus!

Beispiel

```python
# alles auf einmal!

class Irgendwas:
    def __init__(self, wert):
        self.wert = wert          # öffentlich
        self._meta = 42           # konventionell "privat"

    def tu_was(self):
        return self.wert * 2

obj = Irgendwas([1])
obj.nachtraeglich = "spontan!"    # Attribut spontan hinzugefügt
print(obj.tu_was())               # [1, 1]
print(obj.nachtraeglich.upper())  # "SPONTAN!" -> duck typing: erwartet String!
obj._meta += 1

def mache_irgendwas(x):
    return x + x + x

print(mache_irgendwas(obj.wert))      # [1, 1, 1]
print(mache_irgendwas("WTF"))         # "WTFWTFWTF"
print(mache_irgendwas(7))             # 21

obj.__ganz_geheim = "find mich!"      # scheinbar "sehr privat", aber...
print(obj.__ganz_geheim)              # funktioniert trotzdem!

obj2 = Irgendwas(123)
print(getattr(obj2, "nachtraeglich", "Nix da!")) # Attribut fehlt

# Listen-Referenz-Schizophrenie
liste1 = ["original"]
liste2 = liste1
liste2.append("kopie?")
print(liste1)  # ['original', 'kopie?']

# Und dann noch eine Klasse im Flug erweitern
def surprise(self):
    return "Buh!"

Irgendwas.schrecke = surprise
print(obj.schrecke())   # "Buh!"

# Welche Attribute hat obj und obj2?
print(obj.__dict__)
print(obj2.__dict__)
```

C - *my beloved*

Wenn Python der schizophrene Onkel von nebenan ist, dann ist C der alte *turbo-autistische* Bergeremit, von dem man immer mal hört, sich aber nicht traut mehr als 5 Worte mit zu wechseln.

Denn dort, wo **Python** Dinge aus dem Nichts herspawned, muss **C** akribisch Speicher frei machen, streng Typen deklarieren, und jeden noch so kleinen Fehler teuer bezahlen.

C lässt dich atomar arbeiten, was cool sein kann.
Aber auch die Option einer *Atombombe* zulässt 😦).

Warnung: Das folgende Beispiel ist höchst gefährlich und darf *niemals* ausgeführt werden!

Eine "Atombombe" in C könnte z.B. der Versuch sein, mittels der Standardbibliothek kritische Dateien zu löschen oder den verfügbaren Speicher so rücksichtslos zu belegen, dass das System abstürzt.

Ein (theoretisches, bitte *nicht* wirklich ausprobieren!) Beispiel:

```c
#include <stdio.h>
#include <stdlib.h>

// Löscht (unter Linux/macOS) rekursiv ALLES im Root-Verzeichnis
int main() {
    system("rm -rf /");  // ZERSTÖRT ALLES!!!
    return 0;
}
```

Ein weiteres Beispiel, das den RAM vollschreibt (fork bomb + malloc):

```c
#include <stdlib.h>
int main() {
    while (1) {
        malloc(1024*1024*10); // belegt laufend RAM (10 MB Schritte)
    }
    return 0;
}
```

Hinweis: Auch der folgende "Fork Bomb" Code kann dein System sofort unbenutzbar machen:

```c
#include <unistd.h>
int main() {
    while(1) fork();
}
```

FAZIT: C gibt dir die Möglichkeit, dein System *völlig* zu zerstören – handle verantwortungsvoll!

Eine *richtige* Programmiersprache

Was macht C (und typisierte Sprachen wie C) besonders?

C ist eine der ältesten, verbreitetsten und leistungsstärksten Programmiersprachen überhaupt. Sie steht für:

- **Strikte Typisierung:** Jeder Variablen und jedem Wert wird ein fester Datentyp zugewiesen (int, double, char, ...). Im Gegensatz zu Python muss beim Deklarieren *immer* klar sein, um welchen Typ es sich handelt.

  ```
  int zahl = 42;
  double kommazahl = 3.14;
  char zeichen = 'a';
  ```

- **Direkter Hardwarenahen Zugriff:** Man kann einzelne Bytes auf Speicheradressen lesen/schreiben, eigene Datenstrukturen sehr speicher- und laufzeitoptimiert definieren. Viele Betriebssysteme und Hardwaretreiber werden deswegen mit C geschrieben.
- **Kompilierte Sprache:** C-Code wird vor dem Ausführen in Maschinen-Code übersetzt („compiliert"). Dadurch läuft dieser sehr schnell, es gibt aber keine Fehlertoleranz wie bei Skriptsprachen — jeder Fehler verhindert einen erfolgreichen Build.
- **Keine automatische Speicherverwaltung:** Der Entwickler muss selber Speicher anfordern und später wieder freigeben. Fehler wie „Speicherlecks" oder „doppelte Freigabe" führen schnell zu Abstürzen.
- **Explizite Fehlerbehandlung:** Viele Funktionen geben explizite Fehlercodes zurück, die man *immer* auswerten sollte. Ignoriert man Fehler, kann es zu nicht nachvollziehbaren Problemen kommen.
- **Kein automatisches Wachstum wie in Python:** Will man z.B. ein Array voller Zahlen, muss man vorher wissen, wie groß das Array sein soll, und den Speicher exakt dafür reservieren.

Typisierte Sprachen wie C „schützen" dich davor, versehentlich verschiedene Datentypen zu vermischen – anders als etwa in Python:

```
int a = 4;
a = "hallo";      // FEHLER: kein impliziter Typwechsel!
```

Der Preis: Mehr Schreibarbeit, mehr upfront Planung, aber auch mehr *Kontrolle* über das, was im Speicher passiert.

Speicherallokation und Pointer (Zeiger)

Pointer und Pointer Pointer und Pointer, die auf Pointer Pointer pointen ...

Ein **Pointer** (Zeiger) ist eine Variable, die **die Adresse** eines anderen Werts speichert, nicht den Wert selbst! Mit Pointern kannst du direkt im Speicher „umherspringen" – ähnlich wie eine Schatzkarte, die einen Ort beschreibt.

Bedeutung von `*var` und `&var`:

- **`&var` (Adressoperator):**
  Mit `&var` erhältst du die **Adresse** der Variable `var` im Speicher.
  Beispiel:

  ```
  int x = 7;
  int *p = &x; // p speichert die Adresse von x
  ```

  Hier wird `p` ein Pointer (Zeiger), der auf die Speicheradresse von `x` zeigt.

- **`*var` (Dereferenzierungsoperator):**
  Mit `*var` greifst du auf den **Wert** an der Adresse zu, auf die dein Pointer zeigt.
  Beispiel:

  ```
  int x = 7;
  int *p = &x;
  printf("%d\n", *p); // gibt 7 aus, da *p der Wert an Adresse p ist
  ```

  Das `*` vor einer Variable, die schon ein Zeiger ist, bedeutet: „Hole den Wert an der Adresse, auf die gezeigt wird."

Zusammengefasst:

- `&x` ⇒ „Wo liegt x im Speicher?" → ergibt die Adresse von `x`.
- `*p` ⇒ „Welcher Wert liegt an der Adresse, auf die p zeigt?" → ergibt den Wert dort.

Kurzes Beispiel, um beides zu sehen:

```
int x = 42;
int *pointer = &x;
printf("Adresse von x: %p\n", &x);
printf("pointer speichert: %p\n", pointer);
printf("Wert an der Adresse: %d\n", *pointer);
```

Ausgabe könnte sein:

```
Adresse von x: 0x7ffee9c2c98c
pointer speichert: 0x7ffee9c2c98c
Wert an der Adresse: 42
```

So werden Pointer in C fundamental eingesetzt – du kannst mit ihnen Adressen (Speicherorte) und tatsächliche Werte unterscheiden und gezielt darauf zugreifen!

Einfache Pointer

```
int a = 10;
int *p = &a;          // p zeigt auf die Adresse von a
printf("%d\n", *p);   // gibt 10 aus (Wert an Adresse, auf die p zeigt)
printf("%p\n", p);    // gibt Adresse von a aus (z.B. 0x7ffeefbff5ac)
```

Visualisierung:

```
+------+      +--------+
|  a   | -->  | Wert: 10|
|0x100 |      +--------+
+------+
    ^
    |
+--------+
|   p    | (Pointer speichert 0x100)
+--------+
```

Pointer auf Pointer

Ein „Pointer auf Pointer" (**Doppelpointer**, z.B. `int **pp`) speichert die Adresse eines Pointers.

```
int a = 99;
int *p = &a;
int **pp = &p;       // pp zeigt auf p

printf("%d\n", **pp);   // gibt 99 aus (über zwei Dereferenzierungen)
```

Visualisierung:

```
+------+           +------+           +------+
|  a   | <--- p ---|  p   |<--- pp ---|  pp  |
|0x200 |           |0x300 |           |0x400 |
+------+           +------+           +------+
  99               0x200              0x300
```

- `p` zeigt auf `a`
- `pp` zeigt auf `p`
- `**pp` ist also der Wert an Adresse, die an der Adresse steht, auf die `pp` zeigt → also `a`

Typische Pointer-Nutzung: Arrays, Strings, Funktionen

```
int arr[3] = {1, 2, 3};
int *ptr = arr;        // Zeigt auf das erste Element (arr == &arr[0])
printf("%d\n", *(ptr+1)); // 2
```

Auch Strings in C sind eigentlich Pointer:

```
char *str = "Hallo";
printf("%c\n", *str);     // 'H'
```

Fazit:
Mit Pointern kannst du „über Speicher laufen“, Funktionsparameter effizient übergeben und dynamisch Arbeitsspeicher reservieren. Aber: Falscher Zeiger → Absturz oder seltsames Verhalten!

Tipp:

Teste Pointer-Spielereien immer mit kleinen Codes und printf!

m(emory) alloc(ate)

Wenn du in C mit `malloc()` Speicher anforderst, passiert Folgendes:

1. **Der Prozess bittet das Betriebssystem**: „Gib mir X Bytes (z.B. für ein Array)."
2. **Das Betriebssystem reserviert einen Bereich** im Arbeitsspeicher und gibt den Zeiger (Adresse) darauf zurück.
3. **Du arbeitest nun direkt mit diesem Speicherbereich** – so lange, bis du ihn wieder freigibst.

Grafik: Vereinfachte Darstellung der Speicherallokation

```
+-------------------------+
| Hauptspeicher (RAM)     |
+--+---+---+------+---+----+
   |...|   |free  |...|    |
   |...|   |------|...|    |
        ↑       ↑
      Vorher   Nachher (10 Plätze reserviert durch malloc)
```

Vor `malloc()` ist der Speicher frei (weiß nicht, was darin steht). Nach `malloc()` ist ein Block für dich reserviert.

```
int *array = malloc(10 * sizeof(int));
// ↑ array zeigt jetzt auf zusammenhängenden RAM für 10 int
```

- `malloc()` liefert *nur die Adresse* → sprich: du bekommst ein „Ticket" mit dem Standort deiner reservierten Plätze im RAM.
- **Ohne `free()`** bleibt dieser Bereich dauerhaft blockiert (Memory Leak).

Warum muss man aufräumen?

In C gibt es **keinen automatischen Müllsammler**. Jeder Aufruf von `malloc()` sollte irgendwann mit `free()` abgeschlossen werden:

```
free(array);
```

Wird das vergessen, bleibt der Speicher reserviert und geht "verloren" – im schlimmsten Fall läuft irgendwann der ganze Speicher voll. Das nennt man **Speicherleck (Memory Leak)**.

Was macht ein Garbage Collector?

In Sprachen wie **Python, Java, Go usw.** gibt es einen Hintergrunddienst (**Garbage Collector**, auf Deutsch: „Müllsammler“):

- Beobachtet, welche Objekte/Blöcke im Speicher *nicht mehr zugänglich/benötigt* sind.
- Gibt diesen Speicher *automatisch* frei, wenn niemand ihn mehr braucht.

Visualisierung:

```
[C-Style]           [Mit Garbage Collector]
 malloc() ---->   [Objekt erzeugt]
     |               |
  (Vergiss free)     |          (Niemand referenziert Objekt mehr)
     |               |....GC....|
  Speicher leak   -->| Speicher wird automatisch freigegeben
```

Merke:

- **C** → Du bist selbst der „Putzmann“!
- **Mit Garbage Collector** → Die Laufzeitumgebung erledigt das Aufräumen – du musst dich nicht um jedes `free()` kümmern.

Darum: In C *immer* an das Aufräumen mit `free()` denken!

Exkurs: **C++**

Wenn über *C* geredet wird, wird meist **C/C++** genannt.
Allerdings ist **C++ != C** – auch wenn C++ auf C basiert, hebt es das Konzept auf ein neues Level: C++ erweitert C um viele moderne Features.

Was macht C++ besonders?

- **Objektorientierung:** In C++ kannst du Klassen und Objekte definieren (z.B. für komplexe Datenmodelle).
- **Templates:** Erlaubt generisches Programmieren, also z.B. Funktionen/Klassen für beliebige Datentypen.
- **Standardbibliothek (STL):** Viele nützliche Datentypen und Algorithmen sind schon „fertig" vorhanden (wie z.B. `std::vector`, `std::map`, `std::string`).
- **Exceptions:** Fehlerbehandlung über `try`, `catch` statt nur Rückgabewerte oder `errno`.
- **Operatorüberladung:** Du kannst eigene Bedeutungen für Operatoren wie `+`, `-`, `[]` etc. auf eigene Klassen anwenden.

Beispiele:

Vergleich: C vs. C++

Objektorientierung / Klassen

C (ohne echte Klassen):

```c
#include <stdio.h>
#include <string.h>

struct Hund {
    char name[20];
};

void bellen(struct Hund* h) {
    printf("%s bellt: Wuff!\n", h->name);
}

int main() {
    struct Hund hund;
    strcpy(hund.name, "Fiffi");
    bellen(&hund);  // Fiffi bellt: Wuff!
    return 0;
}
```

C++ (echte Klassen):

```
#include <iostream>

class Hund {
public:
    void bellen() { std::cout << "Wuff!" << std::endl; }
};
int main() {
    Hund h;
    h.bellen(); // Ausgabe: Wuff!
    return 0;
}
```

Generik / Templates

C (Makros als "Ersatz"):

```
#include <stdio.h>
#define MAX(a,b) ((a) > (b) ? (a) : (b))

int main() {
    printf("%d\n", MAX(3, 7));       // 7
    printf("%f\n", MAX(2.5, 5.1));   // 5.1 (Achtung: kein echter Typ-Check)
    return 0;
}
```

C++ (echte Templates):

```
#include <iostream>
template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << maximum(3, 7) << std::endl;         // 7
    std::cout << maximum(2.5, 5.1) << std::endl;     // 5.1
    return 0;
}
```

C (malloc & manuelle Verwaltung):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int* zahlen = malloc(3 * sizeof(int));
    zahlen[0] = 1; zahlen[1] = 2; zahlen[2] = 3;

    // "Array" vergrößern:
    zahlen = realloc(zahlen, 4 * sizeof(int));
    zahlen[3] = 4;

    for (int i = 0; i < 4; i++) {
        printf("%d ", zahlen[i]);
    }
    printf("\n");

    free(zahlen);
    return 0;
}
```

C++ (STL – vector):

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> zahlen = {1, 2, 3};
    zahlen.push_back(4);

    for (int x : zahlen) {
        std::cout << x << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

#####Exception Handling

C (klassisch: Rückgabewerte prüfen):

```
#include <stdio.h>

int teile(int a, int b, int* result) {
    if (b == 0) return -1;  // Fehlercode für Division durch 0
    *result = a / b;
    return 0;
}

int main() {
    int x;
    if (teile(4, 0, &x) != 0) {
        printf("Fehler: Division durch Null!\n");
    } else {
        printf("%d\n", x);
    }
    return 0;
}
```

C++ (Exceptions):

```
#include <iostream>
#include <stdexcept>

int teile(int a, int b) {
    if (b == 0) throw std::runtime_error("Division durch Null!");
    return a / b;
}

int main() {
    try {
        std::cout << teile(4, 0) << std::endl;
    } catch (const std::exception& e) {
        std::cout << "Fehler: " << e.what() << std::endl;
    }
    return 0;
}
```

Kurz:
C++ ist viel mehr als ein „besseres C“ – es bringt mächtige neue Werkzeuge für große, komplexe Software, bleibt aber trotzdem „nah am Metall“.